

Bootstrapping

Lecture 14

SEC 595 — Encrypted Computing

- 1 The Ceiling
- 2 Recryption
- 3 Bootstrappable Schemes
- 4 Circular Security

Every scheme so far is levelled

BFV, BGV and CKKS (Lectures 10–12) all carry noise.

- Addition adds noise; multiplication multiplies it.
- Decryption fails once the noise passes a threshold.
- So the multiplicative depth L is fixed at KeyGen.

A ciphertext has a **history**, and its usability depends on it.

A larger modulus raises the ceiling. It never removes it.

$$\text{Dec}_{\mathcal{E}}(\text{Sk}, \text{ct}) = m$$

$\text{Dec}_{\mathcal{E}}$ is fixed, public and finite — the same for every ciphertext.

So it is a valid choice of the function Π in

$$\text{Evaluate}(\text{Pk}, \Pi, \text{ct}_1, \dots, \text{ct}_k).$$

The only obstacle: Evaluate needs its inputs *encrypted*, and one input is the secret key.

The two ingredients

Bootstrapping key

$$\overline{Sk} = \text{Enc}(Pk, Sk)$$

Computed once at setup and **published** with Pk .

Encrypted ciphertext

$$\overline{ct} = \text{Enc}(Pk, ct)$$

A ciphertext is just a string, so it can be a plaintext.

$\overline{ct}$ has two layers: an outer one we compute under, an inner one we want to remove.

Bootstrapping (Gentry, 2009)

$$ct^\# = \text{Evaluate}(\text{Pk}, \text{Dec}_\mathcal{E}, \overline{ct}, \overline{\text{Sk}})$$

No new algorithm: it is `Evaluate` with $\Pi = \text{Dec}_\mathcal{E}$ and inputs $\overline{ct}, \overline{\text{Sk}}$.

Correct: by correctness of `Evaluate`, $ct^\#$ encrypts $\text{Dec}_\mathcal{E}(\text{Sk}, ct) = m$.

The inner layer is stripped, the outer one stays. Nobody sees m , and the evaluator never holds Sk .

Why the result is refreshed

$ct^\#$ comes from one run of Evaluate on the fixed circuit $Dec_{\mathcal{E}}$, applied to fresh $\overline{ct}$ and $\overline{Sk}$.

Its noise depends on $Dec_{\mathcal{E}}$ only — **not** on the history of ct .

Compactness

ct has size bounded in λ (Lecture 8), so $Dec_{\mathcal{E}}$ has fixed size and depth. And $ct^\#$ is an ordinary ciphertext: compactness is preserved.

A worn-out ciphertext returns at a fixed, known noise level.

The condition

A fresh ciphertext supports D levels. Evaluating $\text{Dec}_{\mathcal{E}}$ costs d .

Bootstrappable

$\mathcal{E}$ is **bootstrappable** if it can evaluate its own decryption circuit plus at least one more operation:

$$d < D.$$

After a bootstrap, $D - d \geq 1$ levels remain. Repeat:

evaluate $\rightarrow$ bootstrap $\rightarrow$ evaluate $\rightarrow$ bootstrap $\rightarrow \dots$

The depth is now unbounded: levelled + bootstrapping = **FHE**.

Depth is the currency

The budget is spent by **multiplicative depth**, not gate count.

So Dec_ϵ must be *shallow*, however wide.

Lattice decryption is an inner product and a rounding test — many additions, little depth.

The difficulty: larger parameters raise D , but also enlarge the ciphertext and so raise d . Both grow together.

Bootstrapping is correct but expensive: in practice it dominates the cost.

Example — where the budget goes

A fresh ciphertext supports $D = 10$ levels; $\text{Dec}_{\mathcal{E}}$ costs $d = 7$.

- a. How many useful levels does each bootstrap buy?
- b. How many bootstraps does a circuit of depth 100 need? Check $D + 30(D - d) \geq 100$ and $D + 29(D - d) < 100$.
- c. Give the formula for depth L .
- d. What happens if $d = D$? If $d > D$?

Example — the depth of Regev decryption

Regev decryption (Lecture 7): compute $\Delta = c_2 - c_1^T s \bmod q$ and test whether Δ is near 0 or near $q/2$.

- a. As a circuit on the bits of c_1, c_2, s : where are the multiplications?
- b. Which of them lie in sequence, and which in parallel?
- c. Which part dominates the depth: the inner product or the rounding test?

Example — bootstrapping BGN

BGN (Lecture 9) supports any number of additions and one multiplication. Its decryption computes C^{q_1} , then a discrete logarithm.

- a. Walk through the recipe for BGN. Which step fails?
- b. Is the failure a gap that larger parameters could close?

Encrypting the key under itself

The bootstrapping key $\overline{\text{Sk}} = \text{Enc}(\text{Pk}, \text{Sk})$ encrypts a secret key under its *own* public key.

In the IND-CPA game (Lecture 8, §1.5) the messages are chosen independently of the key. The game says nothing about releasing $\text{Enc}(\text{Pk}, \text{Sk})$.

Circular security

$\mathcal{E}$ is **circular secure** if it stays secure when the attacker is also given $\text{Enc}(\text{Pk}, \text{Sk})$.

Assumed, not proved: no reduction from LWE is known, and no attack either.

Avoiding the assumption

Use a chain of key pairs $(Pk_0, Sk_0), \dots, (Pk_L, Sk_L)$ and publish

$$\text{Enc}(Pk_{i+1}, Sk_i), \quad i = 0, \dots, L - 1.$$

Bootstrapping moves a ciphertext from Pk_i to Pk_{i+1} . No key is ever encrypted under itself.

The price: only L bootstraps, and the public key grows with L . The scheme is **levelled** again.

Unbounded FHE needs circular security.

- 1 Levelled schemes have a fixed depth: noise grows until decryption fails.
- 2 Bootstrapping = $\text{Evaluate}(\text{Pk}, \text{Dec}_{\mathcal{E}}, \text{Enc}(\text{Pk}, \text{ct}), \text{Enc}(\text{Pk}, \text{Sk}))$: decrypt under encryption.
- 3 The output encrypts the same m with noise that depends only on $\text{Dec}_{\mathcal{E}}$, not on the history; compactness is preserved.
- 4 Bootstrappable: decryption depth plus one operation fits the budget, $d < D$. Then levelled becomes fully homomorphic.
- 5 Publishing $\text{Enc}(\text{Pk}, \text{Sk})$ needs circular security; a key chain avoids it but is levelled.